PILLAR DELTA AI

ENTOLI -

Durable Execution as a Typing Problem

Affine Effect Handlers, Non-Persistent Authority, and Recovery as an Ordinary Program

Kyriakos Taraktsis

Pillar Delta AI

Abstract

Systems that let language-model agents act on the world need three guarantees at once: that a computation survives process death, that a tool call is not silently repeated or silently lost, and that an agent cannot exercise authority it was never granted. Existing work supplies the first two through record–replay frameworks and the third through capability-tracking type systems; we are not aware of prior work addressing their interaction, which is where both guarantees quietly fail — a checkpoint captures authority, and a revocation cannot reach a sleeping process.

We present ENTOLI, a language design in which durability, tool invocation, and capability control are consequences of one mechanism: typed algebraic effect handlers with affine continuations. One-shot continuations make durable suspension representable as a uniquely owned stack; an effect-site protocol with exactly two atomic transitions makes every crash dispatch to exactly one recovery action, selected from the durable store alone; and forbidding capabilities from crossing the durable boundary yields *authority at rest*: a persisted continuation confers no authority, so revocation needs no protocol. Recovery itself is an ordinary ENTOLI program. We give the durable machine a formal transition relation and prove that every reachable store satisfies its

invariant, from which recovery-action determinacy follows; the machine's own operations are bound by the same delivery equations as foreign ones, an obligation whose violation we first encountered as a live counterexample in our implementation. A reference interpreter passes cross-process kill tests at every protocol step. In a two-round study, 36 closed-book model-written programs over an adversarial task set produced *zero* violations of the semantic restrictions across 58 generations; all failures were surface-syntax gaps, suggesting the discipline the design imposes is learnable from a one-page specification while the v1 grammar is the present frontier. We state throughout which claims are proved, which are measured, and which remain conditional.

Keywords: algebraic effects, affine continuations, durable execution, capabilities, LLM agents

1 Introduction

An agent program is an unusual object. It runs for days rather than milliseconds. It calls tools that cost money, mutate external systems, and fail. It processes untrusted text that may contain instructions written by an adversary. And it must survive the machine it started on.

Each of these has an established answer in isolation. Long-running computation that survives process death is handled by durable-execution frameworks, whose semantics were given a rigorous account by Burckhardt et al. for Azure Durable Functions [1]. Capability-based confinement of agent tool use has been addressed dynamically by CaMeL, which interposes a custom interpreter and attaches capability metadata to values [2], and statically by Odersky et al., who place agents inside a Scala 3 safety harness using capture checking [3]. Both lines are strong, and this work takes from both.

What is missing is the interaction. The durability literature does not ask what happens to authority when a computation sleeps: capabilities are ordinary values in an ordinary heap, and a checkpoint captures them along with everything else. The capability literature does not ask what happens to a tracked capability when the process holding it dies and is reconstructed four days later from storage. Neither question is idle. A checkpoint that captures a capability is a credential written to disk with an unbounded lifetime, and a revocation that cannot reach a sleeping continuation is not a revocation.

We argue that both problems dissolve if durability, tool access, and authority are consequences of one mechanism rather than three subsystems. That mechanism is typed algebraic effects with handlers [4, 5, 6], read in the capability-passing style of Brachthäuser et al. [7], where an effect type expresses what a computation requires from its context rather than what it does to the world.

Contributions

- A core design in which an effect row is part of every function type, a handler is the only means of discharging an effect, and installing a handler requires holding a capability for it — for every effect, including the suspending effects that reach the outside world (§3).
- An affinity restriction on continuations with two independent justifications: it makes the persisted stack uniquely owned and serialisable in place [8, 9], and it removes the reconciliation problem that multi-shot resumption poses for a durable journal (§4).
- A durable machine with exactly two atomic transitions per effect site, presented not as a second mechanism beside handlers but as *what the machine does to execute a suspending handler clause* (§6); a store invariant; and a metatheory — preservation of the invariant over the full transition relation, from which recovery-action *selection determinacy* follows, with selection computable from the durable store alone (§7, Appendix A).
- *Authority at rest*: capabilities are non-persistable, so revocation needs no protocol and sleeping agents cannot carry stale authority across a policy change (§8).
- A reference interpreter evaluated under hard process death at every protocol step, plus two new studies: a 36-program adversarial model-authorship study with a correction round (58 generations, zero semantic-restriction violations), and performance measurements that quantify the interpreter's two known scale hazards (§10).
- An explicit accounting of which claims are proved, which are measured, and which remain conditional (§11).

2 Thesis

Durable execution is a determinism problem, and determinism is a typing problem. Replay-based durable-execution systems require that code inside a durable region be replayable, and none that we know of can enforce it, because the host language will not discuss the property: nothing in Python or TypeScript prevents a workflow body from reading the clock, and nothing prevents an imported library from opening a socket. Burckhardt et al. make the situation precise: their replay-based model exists specifically so that orchestrations can be written in languages whose runtimes do not support checkpointing [1]. Replay is a workaround for a missing language feature; supply the feature and the workaround goes away with its failure modes.

The property a durable system needs is not only that pure code be replayable, but that *every external effect declare how it recovers*. That declaration is a type-level obligation, and once it exists, recovery-action selection becomes total. The claim is deliberately narrow: a sufficiently

restrictive typed DSL, a compiler plugin, or an effect-aware host language can all reject programs. What a framework cannot do is *propagate* the obligation through an arbitrary call graph, so that a function which transitively performs an unrecovered effect cannot claim to be recoverable. That is what an effect row does, and it is why the mechanism must sit in the type system rather than beside it.

Falsification criterion

ENTOLI earns its existence at three points, and we regard the design as refuted if any fails: a program performing an unrecoverable external effect must fail to compile; a continuation capturing an unserialisable resource must fail to compile; a sub-agent reaching a tool it was never handed must fail to compile. If all three are obtainable inside an existing capability-tracking language, ENTOLI should be a durability extension rather than a language.

The interchange reading

If the intended author is a model (§10), the language's role extends past its own runtime: one agent generates an ENTOLI program from a natural-language request; a checker — not the generating agent — verifies authority and recovery; another agent or runtime executes it; and execution pauses, moves, crashes, and resumes under the machine of §6. The checker is then the hard boundary of the system: agents may propose programs freely, and only valid programs execute. Three properties carry this reading further than intent: the unit of exchange is already defined (a content-addressed typed AST, §8); verification is independent by construction (checking is static and the payload contains nothing opaque); and delegated authority is not a credential (the payload carries a row — a requirement — never a capability, so a received program is inert until the receiver installs handlers from its own authority). What the design does not yet supply — the envelope, authentication beyond content integrity, policy negotiation past a refusal, and any comparative-efficiency claim — we state as open; present evidence supports *more explicit and machine-checkable*, not *more efficient*.

3 The core calculus

3.1 Syntax

$$
\begin{aligned}
\text{values} \quad & v ::= x \mid () \mid \lambda x{:}\tau.\ e \mid \{h\} \mid \kappa \\
\text{expr.} \quad & e ::= v \mid e\ e \mid \mathbf{let}\ x = e\ \mathbf{in}\ e \mid \mathbf{do}\ E.op(v) \\
\mid \ & \mathbf{handle}\ e\ \mathbf{with}\ \{h\}\ \mathbf{using}\ \kappa \mid \mathbf{resume}\ k\ v \mid f\ v \\
\text{handler} \quad & h ::= \mathbf{return}\ x \to e \mid E.op(x; k) \to e,\ h \\
\text{types} \quad & \tau ::= \mathrm{Unit} \mid \tau \xrightarrow{\varepsilon} \tau \mid \tau \multimap_{\varepsilon} \tau \mid \mathrm{Cap}[E, \pi] \mid \mathrm{Res}[\tau, \phi] \\
\text{rows} \quad & \varepsilon ::= \{\} \mid \{E\} \mid \varepsilon \cup \varepsilon \mid \alpha \\
\text{eff. decl.} \quad & ::= \mathbf{effect}\ E\ @s\ \{op{:}\tau \to \tau\ @\delta,\ \ldots\} \\
\text{foreign decl.} \quad & ::= \mathbf{foreign}\ f{:}\tau_1 \twoheadrightarrow \tau_2 \\
\text{susp. class} \quad & s ::= \mathrm{suspending} \mid \mathrm{prompt} \\
\text{delivery} \quad & \delta ::= \mathrm{idem} \mid \mathrm{compensable}(op') \mid \mathrm{alo}(\iota)
\end{aligned}
$$

κ ranges over capabilities; π is a policy (hosts, lane, budget). A foreign name f is *second-class*: not a value, never bound, passed, returned, or abstracted over; it appears only in the application form $f\ v$, which is legal only inside a suspending handler clause and not under a λ within it. $\mathrm{Foreign}[\tau_1, \tau_2]$ classifies declarations only. Capitalised constructor applications build values of the algebraic data universe Data.

3.2 Typing

The judgment is $\Gamma \vdash e{:}\tau/\varepsilon$, with $\Gamma = \Gamma_u; \Gamma_a$ split into unrestricted and affine parts in the standard linear treatment [10, 11]; affine bindings are used at most once.

$$
\mathrm{T-Op} \quad \frac{E.op{:}\tau_1 \to \tau_2\ @\delta \in E \qquad \Gamma \vdash v{:}\tau_1/\{\}}{\Gamma \vdash \mathbf{do}\ E.op(v){:}\tau_2/\{E\} \cup deliv(\delta)}
$$

where $deliv(\mathrm{alo}) = \{\mathrm{AtLeastOnce}\}$ unless an idempotency witness is in scope, and $\{\}$ otherwise: weak delivery is visible in the type and propagates to every caller until deliberately handled — the composition property of §2 in concrete form.

$$
\mathrm{T-Handle} \quad \frac{\Gamma \vdash e{:}\tau/\varepsilon \cup \{E\} \qquad \Gamma \vdash h{:}E \Rightarrow \tau \twoheadrightarrow \tau'/\varepsilon \qquad \Gamma \vdash \kappa{:}\mathrm{Cap}[E, \pi]}{\Gamma \vdash \mathbf{handle}\ e\ \mathbf{with}\ \{h\}\ \mathbf{using}\ \kappa{:}\tau'/\varepsilon}
$$

E is discharged from the row, so a function that handles everything it delegates has an empty row and is pure from outside — the local-purity property of [3], obtained through handlers rather than capture sets. The third premise makes authority unforgeable: writing a handler does not entitle one to install it. The rule governs *every* effect, suspending effects included: there is no other means of discharge. The policy π travels with the installation and is enforced at each clause execution, with budgeted policies spent per execution (§6).

Continuations carry a latent effect row. Capture is deep: the captured continuation runs the remainder of the handled body under the same reinstalled handler, so E performed by the remainder is discharged inside k and $\varepsilon_k = \varepsilon$, the row of the remainder as seen from outside the handler.

$$\mathrm{T-Resume}\quad \frac{\Gamma \vdash k : \tau_2 \multimap_{\varepsilon_k} \tau' \qquad \Gamma \vdash v : \tau_2 / \{\}}{\Gamma \vdash \mathbf{resume}\ k\ v : \tau' / \varepsilon_k}$$

3.3 Handler clauses at the suspension boundary

Effects carry a suspension class: an effect is *suspending* if performing it may yield to the durable machine, *prompt* otherwise (write $Susp$ for the suspending effects). The store invariant of §6 bounds how many operations may be pending, so a handler must not suspend while servicing an operation. The naive rule — requiring $\varepsilon \cap Susp = \{\}$ over the whole clause body, tail resume included — would forbid resumption whenever the ambient row still contains suspending work, since the tail resume contributes $\varepsilon_k = \varepsilon$. The restriction is therefore scoped by position: **resume must appear in tail position, and within the clause judgment the tail resume is typed as a jump, at row** $\{\}$ — its latent effects belong to the remainder and are already accounted at the handle site. (Subtracting ε_k from the body's row does not express this: the subtraction would also erase ambient effects from the body's own row, and with them the very suspensions the rule exists to forbid. Our first formalisation made exactly this error; adversarial review of the rule produced a type-correct program that suspends inside a crash window, §9.1.)

$$\mathrm{T-Clause-Prompt}\quad \frac{\Gamma_u ; \Gamma_a, x : \tau_1, k \vdash e : \tau' / \varepsilon \qquad \varepsilon \cap Susp = \{\} \qquad tail(k, e)}{\Gamma_u ; \Gamma_a \vdash (E.op(x; k) \to e) : E \Rightarrow \tau \twoheadrightarrow \tau' / \varepsilon}$$

$$\mathrm{T-Clause-Susp}\quad \frac{\Gamma_u ; \Gamma_a, x : \tau_1, k \vdash e : \tau' / \varepsilon \qquad \varepsilon = \{\} \qquad tail(k, e) \qquad allpaths(k, e)}{\Gamma_u ; \Gamma_a \vdash (E.op(x; k) \to e) : E \Rightarrow \tau \twoheadrightarrow \tau' / \varepsilon}$$

A suspending clause executes between the machine's two atomic transitions (§6): its body runs with an intent already journalled and its tail resume is the commit, so two further conditions are forced by crash-state analysis rather than taste. *The clause's own row is empty*: the body is pure computation, plus foreign application, plus the tail resume — even a prompt excursion would transfer control out of the crash window with the intent open, creating states the recovery analysis does not classify. *Every control path resumes*: the calculus makes k droppable (abort is a legal exit for prompt clauses), but a suspending clause that aborted would exit the crash window through a third door that is neither atomic transition nor crash — the intent would stay pending

forever while the process runs on, and a later recovery would re-execute the clause, possibly resuming this time and silently reversing the abort. With both conditions the window has exactly two exits, commit and crash, which is what the determinacy theorem needs. Abort is not lost: the foreign call returns a result value, the clause resumes with it, and the *program*, whose continuation is durable, decides.

4 Affine continuations

$k{:}\,\tau_2 \multimap_{\varepsilon_k} \tau'$ is affine: resumable at most once, droppable, never resumable twice. Two independent arguments support the restriction, and we consider the second the stronger. *Representation*: a one-shot continuation is a uniquely owned linked stack that can be walked and written in place, avoiding the copying inherent to multi-shot capture [8]; OCaml adopts the same restriction for the same reason [9]. Since ENTOLI must not merely capture continuations but *serialise* them, the argument is stronger here than in either. *Reconciliation*: a persisted multi-shot continuation resumed twice raises a question with no defensible answer — each resumption performs effects already delivered to the world, so the journal would have to fork into a tree of realised, non-hypothetical futures. Affinity does not solve this; it removes the states in which it can be posed. The cost is that backtracking, generators, and nondeterministic search must be written as explicit trampolines over reified state — acceptable for an orchestration language, unacceptable for a general-purpose one.

Affinity also has a store-level face: *materialisation is affine*. Loading a persisted continuation from the store is itself one-shot per process run, and the encoding preserves sharing — a continuation reachable from two frames persists as one identity and decodes to one cell. Without the sharing clause, affinity silently becomes per-copy: our first encoder flattened shared continuations into independently consumable copies, which is multi-shot resumption with a cold resume in the middle. Across runs the guarantee is deliberately weaker — recovery re-resumes a completed entry's continuation after a crash — and is sound because no committed durable transition is ever repeated (Appendix A).

5 Persistability

Whether a value may cross a durable boundary is three questions, and conflating them is what makes captured foreign resources appear intractable: $Repr(\tau)$ (a byte encoding stable under the value's code identity — decidable), $Reviv(\tau)$ (reconstructible on the far side of a suspension — decidable), and $Perm(\tau, \pi)$ (does policy admit it — a per-deployment judgment). A password is representable and revivable and may still be barred; the separation is load-bearing for §8.

A socket fails $Reviv$; its address does not. The resource is expressed as a description plus a re-acquisition function $\mathsf{Res}[\mathsf{Socket}, \phi]$ with $\phi\colon \mathsf{Addr} \to \mathsf{Socket}/\{\mathsf{Net}\}$, under four conditions that keep ϕ from smuggling the problem inside itself: ϕ is closed (top-level, no free variables); referenced by content hash, never serialised as code; *handled at resume*, so re-acquisition runs under current authority; and prompt, so re-acquisition cannot create a pending entry during resume.

Foreign invokers apply the same discipline to the outside world itself: $\mathbf{foreign}\ f\colon \tau_1 \rightsquigarrow \tau_2$ is closed; persists as its content identity, re-bound by the host at load; the invoker that runs on re-invocation is the one bound in the program in force at recovery (foreign bindings drift under compatibility tokens exactly as handlers do, because they are part of the handler); and a foreign application is not an effect and carries no row — it is the non-atomic world-contact step of the protocol, and the delivery class of the operation whose clause calls it is a *claim about it*, discharged by a conformance harness (§7), never by the calculus. Consumed continuations reachable from later suspensions persist as *tombstones* (identity and consumed flag, no frames), which is what makes the encoder total over reachable configurations rather than well-typed terms only.

6 The durable machine

Configurations are $\langle e \mid K \mid \sigma \rangle$: expression, continuation stack, durable store. A crash may occur between any two transitions; recovery resumes from σ alone. The store is $\sigma = (J, T, F)$: an append-only journal of entries $(j, op, \hat{\delta}, a, \iota, \kappa, s, p)$ — identifier, operation, *the delivery class recorded at intend time*, argument, witness, continuation id, state $\mathsf{P} \mid \mathsf{D}(r)$, parent — a continuation table mapping each κ to its encoded frames, a verification record, a quarantine mark, and a data classification; and an optional final result.

Table 1 The effect-site protocol *is* suspending-clause execution. D-transitions commit atomically; V-transitions do not. A refused or failed D-transition is a no-transition: σ is left exactly as it was.

step	transition = clause execution	kind
1 Intend	entry $(j, op, \hat{\delta}, a, \iota, \kappa, \mathsf{P}, p) + T[\kappa \mapsto enc(K)]$, one transaction, at capture, before the body runs; capability live, budget spent, class recorded	D
2 Invoke	the clause body executes; its foreign application contacts the world — *the crash window*	V
3 Receive	the body's result r, in volatile memory	V

step	transition = clause execution	kind
4 Commit	$\sigma[j := \mathsf{D}(enc(r))]$, a single in-place write, at the clause's tail resume	D
5 Resume	control transfers; affinity gives at-most-once per run	V

The protocol (Table 1) is not a second mechanism beside handlers; it is what the machine does to execute a suspending effect's handler clause, whose shape T-CLAUSE-SUSP already forces. Everything the protocol provides — two atomic transitions, one ambiguous crash case, dispatch on the delivery class — is a statement about clause execution, and everything T-HANDLE provides — installation requires a capability, innermost handler wins, discharge from the row — now governs suspending effects, because they are simply handled effects whose clauses the machine journals around. The regress ("what handles the handler's own contact with the world?") bottoms out at the foreign-function boundary. The continuation is durable *before* any external contact: without a record written before step 2, a crash inside the window leaves no evidence anything was attempted, and recovery cannot distinguish "never started" from "may have completed" [12, 13]. We do not claim to solve the coordinated-attack problem; we claim to make its one ambiguous case dispatchable. The *no-replay* property is stated precisely: a continuation is never re-derived by re-executing pure code — the continuation that receives r is always read from σ. Pure execution forward of a resumed completion is ordinary execution, not replay.

Recording $\hat{\delta}$ at intend time is load-bearing: recovery dispatches on the *record*, never on a re-reading against the program in force, so selection is a function of σ even across a redeployment; if the program in force resolves the entry's operation to a different class, recovery refuses — recoverable, the entry left pending — rather than silently dispatching differently. We adopted this after formal analysis showed the alternative makes the store invariant ill-defined (§9.1).

The store invariant

$\mathrm{INV}(\sigma)$: **I1** (bounded pending) the pending entries form a parent chain of depth at most two, and at depth two the child is exactly the compensation named by the parent's recorded $\hat{\delta}$; **I2** (continuation precedence) every entry's κ resolves in T, written in the same atomic transaction, and never rebound; **I3** (representability) no blob in σ contains capability or host material (structural: the encoder has no arm for them), every blob decodes to what was encoded (assumed as a tested codec-roundtrip property), and each continuation carries its derived data classification. Depth two rather than unbounded is a consequence of a declaration-side condition — a compensation must itself be recoverable by re-invocation alone ($\delta(op') \in \{\mathsf{idem}, \mathsf{alo}\}$) — not an assumption.

Recovery is an ordinary program

Recovery actions perform external effects, so they can crash; if recovery ran on separate machinery it would need its own crash analysis and the determinacy theorem would not reach it. There is no second machine: a recovery action is an effect, performed through the same five-step protocol, with its own journal entry and durable continuation, and the recovery driver is a generated ENTOLI term — inspectable, printable, executed by the same machine as any program. Its privilege reduces to four pure reads of σ plus completion of an already-intended entry, where completion *is* clause re-execution; the capability for this machine-local class is granted by the restart entry point to the driver and is not among the capabilities re-granted to resumed program continuations.

7 Recovery metatheory

Recovery reads σ and acts on its *frontier*: the journal entry of greatest insertion order ($\perp$ if the journal is empty). Step 4's commit updates an entry in place and does not advance the frontier — a definition forced on us by a reachable state our informal phrasing ("the top of the pending stack") did not cover: a crash after a retried operation's commit during a compensable recovery reaches $[\mathtt{D}(op), \mathtt{D}(op')]$ with an *empty* pending stack, and our first implementation wedged there permanently (§9.1). Dispatch is total over what the frontier can be: no entry → run the program entry point; pending → dispatch on the recorded $\hat{\delta}$ (re-invoke for idem/alo with the recorded witness; for $\mathtt{compensable}(op')$, perform op' as an ordinary journalled effect, then re-invoke); done → resume the recorded continuation with the stored result.

Theorem 1 Selection determinacy

For every store reachable in an admissible run, exactly one recovery dispatch is enabled after a crash, and it is computable from σ alone — the final-result slot, the frontier, and the frontier's recorded delivery class. No selection consults state beyond σ; no crash point admits two candidate dispatches.

Theorem 2 Recovery progress

Under the recovery-enabling preconditions (live authority for the re-invoked effect, contract discharged in strict mode, no quarantine or drift on the materialised continuation, the program in force agreeing with the recorded class), with finitely many crashes and terminating clause bodies: each crash-free recovery pass either refuses — leaving σ unchanged up to one sticky quarantine mark, and re-runnable — or reaches program re-entry after a bounded prefix of at most one child intend and two commits, repeating no committed durable transition.

Both rest on preservation of INV over the full transition relation (Appendix A), proved by mutual induction with a window-discipline lemma; on a machine-local conformance lemma — *the machine's own operations are bound by the same delivery equations as foreign ones*, so completion of an already-completed entry returns the stored result rather than refusing — and on an unconditional no-repeat lemma. The conformance lemma is not an aesthetic: its violation was a live Theorem 1 counterexample in our implementation, found by review one crash deeper than our own test matrix (§9.1).

What the theorems do not establish

They select and complete recovery *actions*; end-to-end exactly-once at the world is conditional on the declared delivery contracts, which are claims about foreign systems: idem requires $op; op \approx op$; $\mathsf{alo}(\iota)$ requires $op(\iota); op(\iota) \approx op(\iota)$; and $\mathsf{compensable}(op')$ requires $op^a; op'; op^b \approx op$ for $a \in \{0,1\}, b \geq 1$ — strengthened from the naive $op; op'; op \approx op$ because the compensation fires in the ambiguous case where op may never have been delivered ($a = 0$), and a crash during the retry's own window re-invokes without a fresh compensation ($b > 1$). These obligations are discharged by a conformance harness that generates a test per declared class and refuses unproven contracts in strict mode: testing, not proof, but named, localised, and reviewable — roughly where unsafe blocks live in practice. This is the ordinary shape of a foreign-function boundary, inherited from the Saga discipline [14] and idempotency-key practice [15].

8 Authority at rest

> **Rule (no persisted authority).** $Perm(\mathsf{Cap}[E, \pi], \cdot)$ is false for all E, π: capabilities never cross a durable boundary.

At suspension the continuation splits as $K = K_{\mathrm{frames}} \otimes K_{\mathrm{caps}}$ and only K_{frames} is written; a capability anywhere in the encoded state — in a handler frame or first-class in an environment — persists as an *effect-name slot*: which effect was handled, never the authority to handle it. At resume (and at recovery, whose restart prelude is an ordinary sequence of handle installations from the entry point's own authority), slots re-bind against the grants in force *now*; a slot with no matching installation is a refusal naming the effect. Revocation is therefore not an operation but an absence, and budgets bind at the figure in force at resume, not at suspension.

Theorem 3 Authority at rest

For every reachable store, no blob or field contains capability material; and a resumed computation holds precisely the authority granted by the handlers in force at resume time, never the authority in force at suspend time.

The formal half is structural (the encoder has no arm for capabilities; every store write passes through it), plus one closure fact: no transition derives authority from the content of σ. Neither property is obtainable by adding durability to capture checking [3], which has no notion of a boundary across which a capture set must be discharged, nor by adding capabilities to replay [1], which reconstructs the capability with the rest of the state. The theorem says nothing about secrets: a persisted frame may legitimately hold confidential data that policy admits, so a store's handling classification is *computed* as the join of what it holds, never asserted.

Code identity and resume compatibility

Definitions are named by a Merkle hash [16] over the typed AST, closed over everything referenced (strongly connected components hashed as groups for mutual recursion); names with *no* definition at persist time are recorded with an absence sentinel, so a definition *appearing* is drift symmetric with one changing. Effect signatures hash operation argument and result types along with delivery classes — we found and repaired the type half missing in our implementation, which had let a type-only redeployment resume a stale continuation undetected. Signature equality is still not semantic equality: a four-day-old continuation may resume against a handler with an identical signature that now routes to a different merchant. What must be stable is the observable contract the continuation depends on — a third thing the compiler cannot compute — so it is *declared* as a compatibility token beside the computed hash, and both halves are required at resume; a mismatch quarantines the continuation durably rather than losing it. Verification is dependency-precise in both directions: a sleeping continuation is stranded by a change to what it still references, and by nothing else.

9 Implementation

The reference interpreter is a Rust workspace of nine crates (syntax, CEK-style evaluator, closed codec with content addressing, SQLite-backed durable store, runtime, generated recovery driver, static checker, test kit, CLI), 79 tests. Continuations are explicit heap frames; the encoder covers every value form except capabilities, which have no byte form by construction. Until the checker is total, every static guarantee carries a dynamic proxy: one-shot resume is a consumed flag plus a per-run materialisation set; representability is the closed encoding; the window discipline is a machine refusal of operations inside the window; pending depth carries a runtime assertion. Crash tests use hard process death — a subprocess aborted at a named protocol step, restarted from cold durable state — never in-process exceptions, and remote fixtures record their own state independently.

9.1 Methodology adversarial review as a falsification instrument

Beyond conventional testing we subjected the design's artifacts — the paper text, the typing rules, and the metatheory — to multi-round adversarial review by independently prompted model agents (16–34 per round) instructed to construct counterexamples, with each finding re-verified by a second skeptical agent before we accepted it. We report this as methodology, not as peer review: the agents were commissioned by us. The instrument earned its place by finding, among others: a reachable crash state one step deeper than our own test matrix, in which the machine's privileged completion violated its own declared idempotency equation (a live counterexample to Theorem 1, repaired by the conformance lemma of §7); a formalisation of the clause side condition that type-checked a program suspending inside its own crash window (repaired by the jump-typed tail resume of §3.3); first-class foreign invokers smuggling world contact past the crash window under a λ (repaired by making foreign names second-class); and the recovery-dispatch drift repaired by recording delivery classes in the journal (§6). Each confirmed finding was closed with a regression test the same day, and the full register — including two recorded dynamic-proxy gaps that remain open for unchecked programs — ships with the artifact.

10 Evaluation

10.1 Crash matrices and demonstrations

Four demonstrations, each tied to a falsification clause, all passing under hard process death. *Kill-and-resume*: an approval/settlement flow killed at each of four protocol points yields exactly one ledger entry per run, restarted from cold storage; the compensable variant's matrix includes the crash-after-retry-commit row that wedged the pre-repair machine. *Containment*: a delegated computation whose row is not discharged fails to compile, naming both the escaping effect and the missing capability — for the whole language including suspending tools, so a delegate can reach no tool beyond its envelope whether or not the process binds one. *Authority at rest*: a byte-level scan of the persisted continuation finds no capability or budget material; revocation while sleeping (the restart prelude simply does not install the handler) refuses resume naming the effect; re-granting under a smaller budget binds the resumed agent to the smaller figure, and completed inferences are never re-run. *Delivery-drift*: a redeployment changing a pending operation's delivery class refuses recovery with the entry left pending; the unchanged program recovers through the recorded dispatch.

10.2 Can a model write ENTOLI

The restrictions ENTOLI imposes — affinity, tail-position resumption, mandatory delivery classes, witnesses — are tedious for a person and free for a checked generator, which raises the design's real usability question: can a model write it from a specification alone? An earlier calibrated pilot (six fresh-context instances, one-page reference card, tasks written by us in the card's own idiom) produced 6/6 first-pass-compiling programs. That result is encouraging and weak: the tasks were calibrated to the card's worked examples by the card's author.

We therefore ran an adversarial version with the author bias removed from task selection: a model agent, given only the card, authored 12 tasks explicitly targeting the corners where the restrictions bite (non-tail temptations, double resumes, witness derivations, compensation choice, abort semantics, continuation-typed helpers); 36 fresh-context, closed-book solver instances across three model tiers (12 tasks × {Haiku, Sonnet, Opus}) wrote one program each; every program was compiled by the real checker; failures received exactly one correction round with the verbatim compiler diagnostics.

Table 2 Adversarial authorship study: 12 model-authored tasks, closed book, one-page specification. "Fixed" = compiles after one round of verbatim compiler diagnostics.

model tier	first pass	fixed (+1 round)	cumulative
Haiku	5/12	+0	5/12
Sonnet	4/12	+3	7/12
Opus	5/12	+3	8/12
total	14/36 (39%)	+6	20/36 (56%)

The aggregate rates (Table 2) are far below the calibrated pilot — and the *distribution* of failures is the result. Across all 58 generations (36 first-pass + 22 corrections), the semantic restrictions were violated **zero** times: no double resume, no non-tail resume, no suspension inside a window, no missing witness (E003/E004/E005 never fired). Every failure was a surface gap: 17 parse errors concentrated on grammar corners the v1 parser does not implement (parenthesised continuation types in signatures, parenthesised **resume** arguments, nested handle expressions, if under operator contexts) and 5 type errors from one specification gap (the card documents no string equality, and the language surfaces none, so tasks branching on string content were unsolvable as stated — two correction-round agents diagnosed this gap explicitly and correctly in

comments). The discipline the design exists to impose was learned from one page by every tier; the thin v1 grammar, not the semantics, is the measured frontier.

Caveats, stated plainly. The metric is compilation, not functional correctness: at least three correction-round programs satisfied the checker by weakening the task's semantics (answering constants and compensating in the return clause), which compilation cannot see — functional grading needs executable task oracles, which we leave as the next study. Task author and solvers are one vendor's models (three distinct tiers); a cross-vendor replication remains to be run. Sample sizes are modest. We publish all 12 tasks, 58 programs, and diagnostics with the artifact.

10.3 Performance of the reference interpreter

Table 3 Measured costs (release build, Apple Silicon, SQLite store, minimum of three runs). Left: per-effect cost as the journal grows (N sequential effects). Right: encoded continuation size as captured depth grows (D live frames with accumulating environments).

N	ms/effect	store	D	blob bytes	ratio
25	2.1	53 KB	10	1 355	—
50	2.8	123 KB	25	6 140	4.5×
100	4.7	414 KB	50	22 115	3.6×
200	9.5	1.2 MB	100	84 065	3.8×
400	20.5	4.3 MB	200	333 215	4.0×

Table 3 quantifies the interpreter's two known scale hazards. Per-effect cost grows roughly linearly with journal length (quadratic total cost over a run): the mechanism is measured and understood — the runtime re-verifies the store invariant by a whole-journal scan after every durable transition, and journal-id allocation scans the journal again. This audit is exactly what the preservation theorem (Appendix A) licenses retiring from release builds; with indexed counts it becomes $O(1)$ per effect. Encoded continuation size grows quadratically with captured depth ($4\times$ bytes per $2\times$ depth): environments are encoded without structure sharing, so D live frames with accumulating bindings encode $\Theta(D^2)$ bytes — a continuation of depth 200 persists 333 KB at every effect site. The repair (back-references for shared environment chains, as the encoding already does for shared continuations) changes no semantics. We report these as costs of the *reference interpreter*, not of the design, with mechanisms identified and repairs scoped; a

comparative evaluation against replay-based engines is future work and no efficiency claim is made here.

11 Status of claims

So that nothing is read as more settled than it is: effect-row discharge and local purity are inherited from known results [4, 6, 7]. Preservation of the store invariant, selection determinacy (Thm. 1), the machine-local conformance lemma, the no-repeat lemma, and the store half of authority at rest (Thm. 3) are *proved* over the transition relation of Appendix A, assuming encoder totality over reachable configurations (demonstrated for the interpreter's value domain, open in general) and a tested codec-roundtrip property; for checked programs they additionally rest on the standard substitution/preservation lemmas of the calculus, which we state with sketches and have not mechanised. Recovery progress (Thm. 2) is proved under stated hypotheses (finitely many crashes; terminating clause bodies). End-to-end exactly-once is *not claimed*: it is conditional on delivery contracts discharged by testing. Resume compatibility is a design decision whose soundness depends on deployers bumping tokens honestly. Serialisable continuations are *demonstrated* for the interpreter's closed value domain under hard process death — cross-process cold resume with sharing preserved — and remain open for host-language value domains, which is the design's weakest premise and the shortest path to refuting it.

12 Limitations and future work

The grammar, not the semantics, failed our hardest usability test (§10.2); completing the surface language is engineering, now prioritised by the study's concrete failure list. Delivery contracts are tested, not proved, and a handler can pass a sandbox and fail in production. Compatibility tokens depend on deployer discipline; the mechanism makes the obligation visible, not inevitable. Nested suspension is forbidden rather than solved, and suspending clauses additionally forbid prompt excursions — even a log line must live outside the window — with a candidate relaxation (prompt handlers installed inside the clause) awaiting crash-state analysis. Concurrency is absent, not deferred cleanly: the design fixes only the commitments that would be unmigratable later (hierarchical continuation identity, a partially ordered journal, affine budgets that split at fork, cancellation as revocation at a subtree root). Cost and deadline obligations are typed but only partially provable, following [17, 18]. Two dynamic-proxy gaps for *unchecked* programs are recorded and open. The functional-correctness study (executable oracles), a cross-vendor authorship replication, mechanisation of the calculus lemmas, and the comparative evaluation against replay engines are the immediate next work.

13 Related work

Algebraic effects and handlers [4, 5] with row typing [6] supply the mechanism; Brachthäuser et al.'s capability reading [7] is what lets one mechanism serve durability and authority at once. Object-capability security [19, 20] and least privilege [21] supply the discipline; Odersky et al. apply it to agents via Scala 3 capture checking [3], settling the static-capability question for live processes — §8 is what that model becomes when the process is allowed to die. CaMeL [2] achieves comparable confinement dynamically, evaluated on AgentDojo [22]; our containment claim is deliberately narrower than its information-flow goals, and indirect prompt injection [23] is addressed here only as envelope escalation. Durable Functions [1] gives replay-based durability a rigorous semantics; we remove replay's preconditions rather than its formalisation. Compensations descend from Sagas [14], idempotency keys from [15], the impossibility bound from the coordinated-attack problem [12, 13]. Continuation persistence draws on web-server continuations [24], orthogonal persistence [25], and one-shot representation [8, 9].

14 Conclusion

Durable execution and capability safety have been pursued independently and successfully; their interaction is where both guarantees fail, and ENTOLI derives both from one mechanism. The two headline properties — recovery-action selection computable from the durable store alone, and authority that cannot outlive its justification — are now theorems over a formal machine whose reference implementation survives being killed at every step of its own protocol. The sharpest empirical finding is that the design's discipline, built to be checkable rather than ergonomic, was never once violated across 58 closed-book model generations on adversarial tasks: the checked generator the design anticipates appears to exist, and what stands between this design and a production system is engineering whose costs are measured, not open semantics.

Acknowledgments

The design was hardened by multi-round adversarial review performed by independently prompted model agents (§9.1); all confirmed findings and their regressions ship with the artifact.

A The durable machine formally

This appendix condenses the metatheory; the full development (every rule, every case) accompanies the artifact.

Stores and configurations

$\sigma = (J, T, F)$ as in §6; T maps κ to (blob, verification record H — the $(E, sig(E), compat(E))$ set recorded at persist time plus the code-identity header with absence sentinels — quarantine mark q, classification ℓ). Configurations are running $\langle e \mid K \mid \sigma \rangle_v$ (volatile state v: grant table, materialised set, contract set, program-and-compat in force), crashed $\langle \mathsf{crash} \mid \sigma \rangle$, and final. Crash is a transition available from every running configuration (and from final, before observation) that forgets everything but σ. *Admissible runs* start from the empty store, and in each restart segment the program in force is checked, or the run is guarded by the dynamic proxies; no cross-restart program stability is assumed — delivery drift meets the recorded-$\hat{\delta}$ refusal, signature and compatibility drift meet the materialisation guard, and both refusals are no-transitions.

Transitions

D-transitions: INTEND (entry with recorded $\hat{\delta}$ + encoded continuation, one transaction; with a handler present the capability, budget, and strict-contract checks are preconditions of the whole transaction; without one the write still happens and the run halts awaiting restart — the machine is deliberately permissive so continuations exist below the runtime); COMMIT (one in-place pending→done write at the window's tail resume); QUARANTINE (one sticky mark on a materialisation whose verification record disagrees with the program or token in force); COMMITFINAL (write-once). V-transitions: pure steps, window execution with foreign application, resume, materialisation (one-shot per run per identity, guards first), and the recovery dispatches (REC-FINAL, REC-EMPTY, REC-DONE, REC-COMPLETE, COMP) — where COMP's child captures, by construction of the generated driver, the driver's own frame whose next act is completion of the parent (Assumption D, pinned by the crash matrix). No transition removes an entry or cell, reorders the journal, rewrites a blob, or changes a committed state.

Lemma 1 Window discipline

In every admissible run, a root INTEND fires only with no entry pending, and a child INTEND (a compensation) fires only with exactly its parent pending.

Theorem 4 Preservation

Every store reachable in an admissible run satisfies $\mathrm{INV}(\sigma)$.

Proof shape (mutual strong induction with Lemma 1 on run length). The lemma's case analysis enumerates every door through which program execution is re-entered after a crash — empty journal; final result; done root (any pending entry would already have been pending at that root's intend, contradicting the induction hypothesis, since states move only pending→done); done child with parent pending (the resumed frame is the driver's, which completes the parent before program re-entry); done child with parent done (the crash-after-retry-commit state: pending is already empty and completion takes the stored-result branch); completed re-invocations of a root or of a compensation child — and each door hands program execution an empty pending set. Preservation then checks each D-transition: INTEND preserves I1 by the lemma at that step and I2 by transactionality and freshness; COMMIT, QUARANTINE, and COMMITFINAL shrink or leave the pending chain and touch no reference. The case that cannot arise — an intend from inside a window, which would break the depth bound — is excluded by the empty-own-row clause condition (checked) or the window-refusal proxy (unchecked); the case a compensable *chain* would create is excluded by the declaration-shape condition at every program load. ∎

Lemma 2 Machine-local conformance

Completion of an already-completed entry returns the stored result with no durable transition; completion of a pending entry performs one clause re-execution and at most one commit per attempt, and a post-crash repeat lands in the first branch if the commit landed. At the store, completing an entry $n \geq 1$ times equals completing it once. At the world, each re-execution is a real re-invocation bounded only by the operation's declared equation — the conditional obligation of §7.

Lemma 3 No repeat

No durable transition re-performs a committed one: commits on completed entries never fire (Lemma 2 precedes them), intends are fresh, quarantine marks are per-cell one-way, the final result is write-once.

Theorem 1 follows from preservation, frontier totality, and totality of the recorded-class dispatch (every journalled operation carries a class, because intending an undeclared operation refuses); Theorem 2 from Lemmas 1–3 by induction over crash-free passes, with monotone progress across crashes (append-only journal with at most one child per parent, states pending→done only, marks one-way); Theorem 3's store half by induction on the run, since every blob enters σ through the encoder, which maps any capability to an effect-name slot and has no other arm for it, plus the closure fact that no rule extends the grant table from σ's content. Known correspondence divergences between this relation and the implementation (the runtime does not itself invoke load-time validation; the budget debits before the intend transaction; final-commit

divergence is ignored; two dynamic-proxy gaps for unchecked programs) are recorded in the artifact's audit and do not affect checked programs. ∎

Citation identifiers

[1] burckhardt2021durable

[2] debenedetti2025camel

[3] odersky2026securing

[4] plotkin2009handlers

[5] bauer2015programming

[6] leijen2017type

[7] brachthauser2020effects

[8] bruggeman1996representing

[9] sivaramakrishnan2021retrofitting

[10] girard1987linear

[11] wadler1990linear

[12] gray1978notes

[13] halpern1990knowledge

[14] garcia1987sagas

[15] helland2007life

[16] merkle1987digital

[17] hoffmann2012multivariate

[18] lucassen1988polymorphic

[19] dennis1966programming

[20] miller2006robust

[21] saltzer1975protection

[22] debenedetti2024agentdojo

[23] greshake2023not

[24] queinnec2000influence

[25] atkinson1995orthogonal